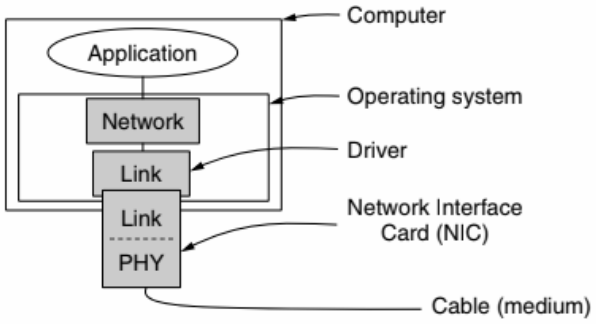
ELEMENTARY DATA LINK PROTOCOLS

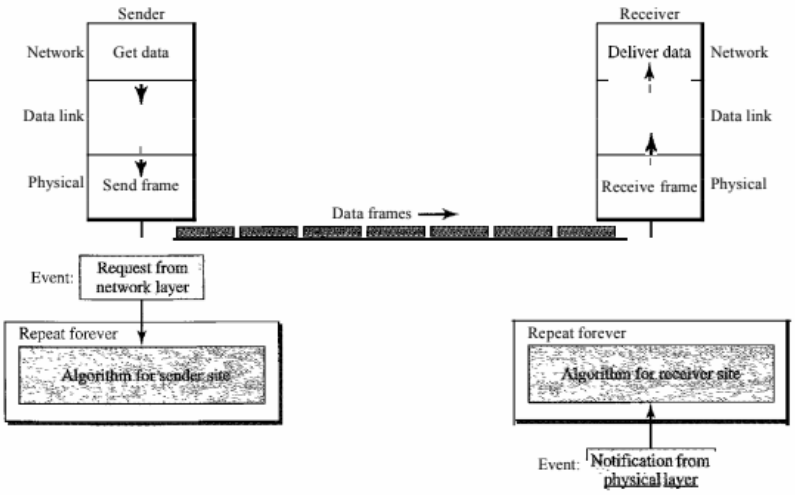
The physical layer process and some of the data link layer process run on dedicate hardware called a NIC (Network Interface Card). The rest of the link layer process and the network layer process run on the main CPU as part of the operating system, with the software for the link layer process often taking the form of a device driver.



1. A Utopian Simplex Protocol
2. A Simplex Stop-and-Wait Protocol for an Error-Free Channel
3. A Simplex Stop-and-Wait Protocol for a Noisy Channel

A Utopian Simplex Protocol

A **Utopian Simplex Protocol** is the simplest data link layer protocol in computer networks. It is called *Utopian* because it assumes a perfect communication channel where nothing goes wrong. Therefore, it does not need error control or flow control**.**



The protocol assumes:

- Data flows in only one direction — from sender to receiver (simplex).
- No processing delay needs to be considered. There is infinite buffer space.
- The communication channel never loses or damages frames. Therefore, no acknowledgements, sequence numbers, error control, or flow control are required.

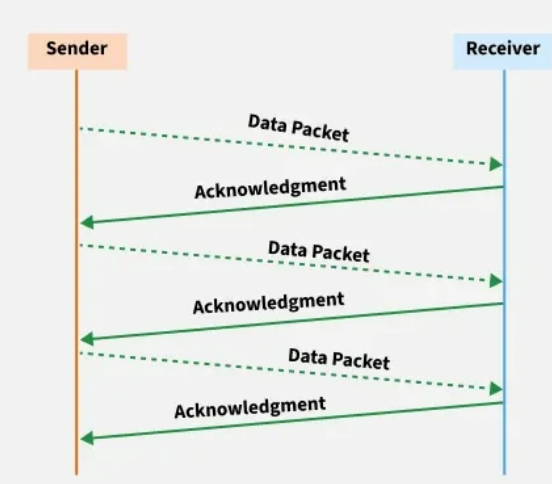
The sender sends frames continuously, and the receiver accepts them continuously. Since the channel is assumed to be perfect, there is no need to ask *"Did you receive my frame?"* or *"Was the frame damaged?"*

A Simplex Stop-and-Wait Protocol for an Error-Free Channel

A **Simplex Stop-and-Wait Protocol** is a data link layer protocol in which data is transmitted in only one direction. The sender sends **one frame at a time** and waits for an **ACK** before sending the next frame. It assumes that the channel is error-free, so frames are not lost or damaged. **Flow control is required** to prevent the sender from sending data faster than the receiver can process. Thus, the ACK helps control the flow of data and ensures that the receiver is ready for the next frame.

Working of A Simplex Stop-and-Wait Protocol

1. The **sender** gets a packet from the network layer.
2. It creates a **frame** and sends it to the receiver.
3. The sender **stops and waits** for an acknowledgement (ACK).
4. The **receiver** receives the frame and passes its data to the network layer.
5. The receiver sends an **ACK** to the sender.
6. After receiving the ACK, the sender sends the **next frame**.
7. This process continues until all data is transmitted.



A Simplex Stop-and-Wait Protocol for a Noisy Channel

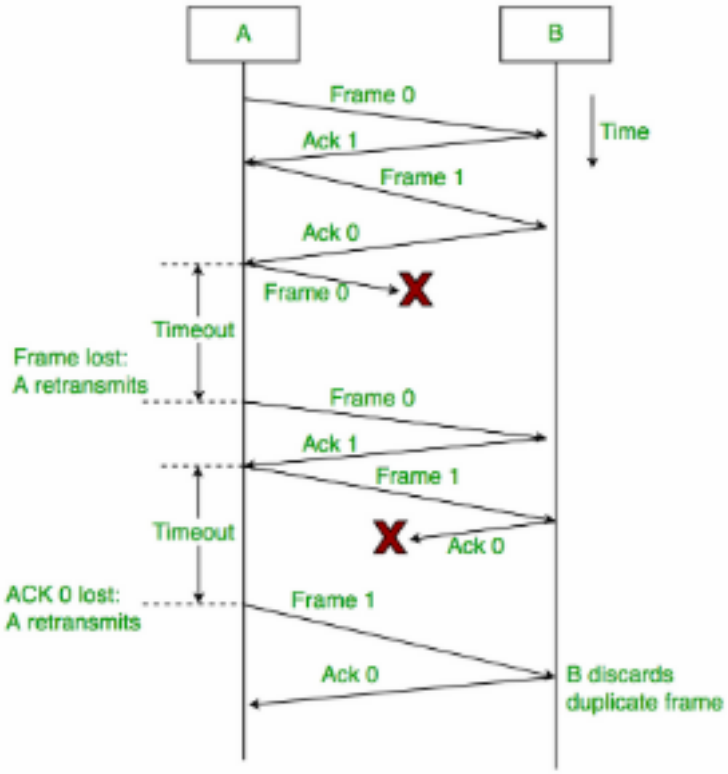
A **Simplex Stop-and-Wait Protocol for a Noisy Channel** is a data link layer protocol in which data is transmitted in only one direction over a channel where frames may be **lost or damaged**. The sender sends **one frame at a time** and waits for an **ACK** before sending the next frame. A **timer** is started when the frame is sent; if the ACK does not arrive before the timer expires, the sender assumes that the frame or ACK was lost and **retransmits the frame**. Thus, the protocol provides both **flow control and error control**.

The sender sends one frame and waits for an ACK. If the ACK is received correctly, it sends the next frame. If the ACK is damaged or the timer expires, it **retransmits the same frame**. The sequence number helps the receiver identify and reject duplicate frames.

Protocols in which the sender waits for a positive acknowledgement before advancing to the next data item are often called **ARQ (Automatic Repeat reQuest) or PAR (Positive Acknowledgement with Retransmission).**

Working of Simplex Stop-and-Wait Protocol for a Noisy Channel

1. The **sender sends a frame and starts a timer**.
2. The sender then waits for one of three events:
    1. A **valid ACK** arrives.
    2. A **damaged ACK** arrives.
    3. The **timer expires**.
3. If a valid ACK arrives, the sender:
    - Gets the **next packet** from the network layer.
    - Updates the **sequence number**.
    - Sends the next frame.
4. If the ACK is **damaged** or the **timer expires**, the sender does not change the packet or sequence number. It **retransmits the same frame**.
5. When the receiver gets a frame, it checks the **sequence number**:
    - If it is a **new frame**, the receiver accepts it, passes the data to the network layer, and sends an ACK.
    - If it is a **duplicate or damaged frame**, the receiver does not pass it to the network layer. Instead, it sends an ACK for the **last correctly received frame**.



In stop and wait method where a sender sends one frame and waits for an ACK, a 1-bit sequence number (0 or 1) is sufficient. At each instant of time, the receiver expects a particular sequence number next. When a frame containing the correct sequence number arrives, it is accepted and passed to the network layer, then acknowledged. Then the expected sequence number is incremented modulo 2 (i.e., 0 becomes 1 and 1 becomes 0). Any arriving frame containing the wrong sequence number is rejected as a duplicate. However, the last valid acknowledgement is repeated so that the sender can eventually discover that the frame has been received.

In earlier protocols, data was transmitted in only one direction. In practical networks, however, data needs to be transmitted in both directions, known as full-duplex communication. A more efficient approach is to use the same link for transmitting both data and acknowledgements. The receiver identifies whether an incoming frame is a data frame or an acknowledgement frame by checking the kind field in the frame header.

Sliding Window Protocols : In a sliding window protocol, every data frame has a **sequence number**. The sequence numbers usually range from **0 to $2^n - 1$**, where *n* is the number of bits used for the sequence number. For example, with **n = 1**, the sequence numbers are only **0 and 1**, as used in Stop-and-Wait.

Sending Window and Receiving Window : The main idea of a sliding window protocol is that both the **sender and receiver maintain a window** and the sender and receiver windows **do not have to be the same size**.

Sender window: The sequence numbers within the sender's window represent frames that have been sent or can be sent but are as yet not acknowledged. Whenever a new packet arrives from the network layer, it is given the next highest sequence number, and the upper edge of the window is advanced by one. When an acknowledgement comes in, the lower edge is advanced by one. In this way the window continuously maintains a list of unacknowledged frames.

Receiver Window: The receiving data link layer's window corresponds to the frames it may accept. Any frame falling within the window is put in the receiver's buffer. When a frame whose sequence number is equal to the lower edge of the window is received, it is passed to the network layer and the window is rotated by one. Any frame falling outside the window is discarded.

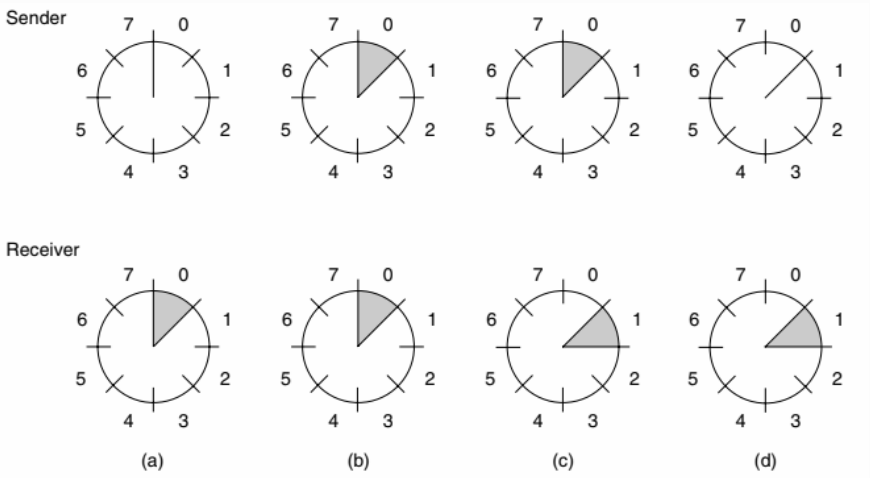


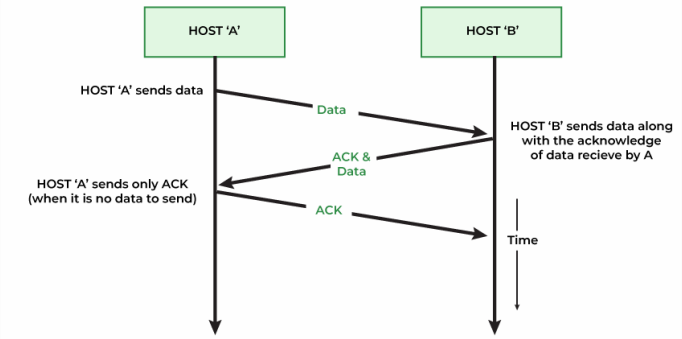
Figure 3-15. A sliding window of size 1, with a 3-bit sequence number. (a) Initially. (b) After the first frame has been sent. (c) After the first frame has been received. (d) After the first acknowledgement has been received.

Figure 3-15 shows an example with a maximum window size of 1. Initially, no frames are outstanding, so the lower and upper edges of the sender's window are equal, but as time goes on, the situation progresses as shown. Unlike the sender's window, the receiver's window always remains at its initial size, rotating as the next frame is accepted and delivered to the network layer.

Piggybacking

Piggybacking : Piggybacking: A more efficient method where data and acknowledgements share the same channel. The receiver delays sending the ACK and attaches it to the next outgoing data frame. The sender identifies data and ACK information using the frame header, reducing the number of separate control frames and improving overall network efficiency. Piggybacking makes better use of the available channel bandwidth by combining data and acknowledgment into a single frame, eliminating the need to send separate ACK frames. This reduces control overhead, lowers transmission costs, and improves the overall latency of data transfer, especially in bidirectional communication

.



As we can see in the figure, we can see with piggybacking, a single message (ACK + DATA) over the wire in place of two separate messages. Piggybacking improves the efficiency of the bidirectional protocols.

- If a host has both data and an acknowledgment, it sends a single frame containing both.
- If only an acknowledgment exists, the host waits for some time to see if data is ready; otherwise, it sends the ACK separately.
- If only data is left, the last acknowledgment is included in the data frame.

Pipelining

Pipelining is a technique in which a sender can send multiple frames before receiving an acknowledgment (ACK) for the previous frame. Pipelining improves transmission efficiency, especially when the **bandwidth-delay product (BDP)** is large. The bandwidth-delay product represents approximately how many bits can be present in the network while a packet is traveling from sender to receiver.

For example, if: Bandwidth = **10 Mbps** and Round-trip time = **100 ms** Then:

$$\textbf{BDP = Bandwidth} \times \textbf{RTT} = 10{,}000{,}000 \times 0.1 = \textbf{1,000,000 bits}$$

So, if we use Stop-and-Wait, much of the available network capacity may remain unused while waiting for ACKs. Pipelining keeps the link busy by transmitting multiple frames before waiting for acknowledgments.

Some of the sliding window protocol are

1. One-bit sliding window protocol
2. Go-Back-N Automatic Repeat Request
3. Selective repeat

1.

One-bit sliding window protocol

The **One-Bit Sliding Window Protocol** is a simple **flow-control and error-control protocol** used at the data-link layer. It is a sliding window protocol with a window size of 1 thus is uses a **1-bit sequence number**, so the sequence numbers are only **0 and 1**. The sequence numbers are based on modul-2 arithmetic.

The **one-bit sliding window protocol for bidirectional communication** allows both computers, A and B, to send data frames to each other at the same time. Each frame contains a sequence number (0 or 1)**,** an acknowledgement number**,** and data. When A sends a frame, B receives it and sends back its own data frame containing an acknowledgement for A's frame; similarly, A acknowledges B's frame in its next data frame. This is called **piggybacking**, where the acknowledgement is carried along with the data instead of sending a separate ACK frame. The sequence numbers alternate between 0 and 1 to identify new frames and detect duplicates. If a frame or acknowledgement is lost, the sender waits for a timeout and retransmits the same frame. When the correct frame is received, the receiver accepts it, delivers the data to the network layer, and changes its expected sequence number. Thus, the protocol provides reliable two-way communication, detects duplicate frames, and retransmits lost frames**.**

Each frame is written as: **(Seq, ACK, Data)**

For example: **(0, 1, A0)** means:

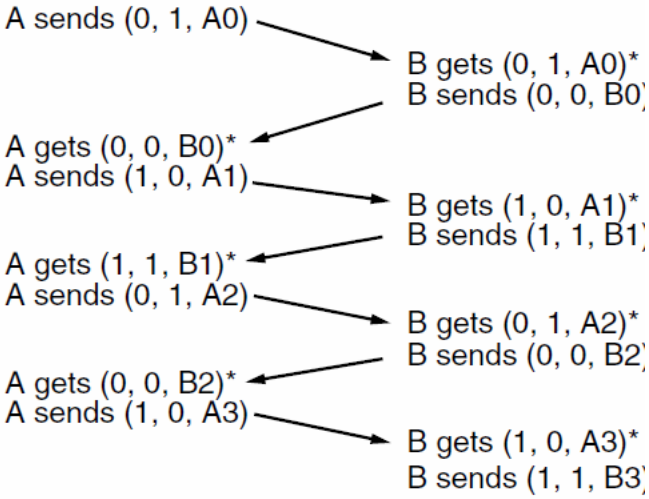
- `0` → sequence number of A's frame
- `1` → ACK number carried by A's frame
- `A0` → data from A

The sender:

1. Sends a frame with sequence number **0**.
2. Waits for an acknowledgment. **ACK field** tells the sender the **last frame received correctly**. If the ACK **matches the sender's frame sequence number**, the sender knows the frame was successfully received.
    1. After receiving the correct ACK, the sender then **gets the next packet** from the network layer sends the next frame with sequence number **1**.
    2. If the ACK **does not match**, the sender **retransmits the same frame**.
3. This continues alternately: **0 → 1 → 0 → 1... sequence number for frame**

The Receiver: **Every received frame generates a response/ACK frame**, even if the received frame is a duplicate.

Example: A Scenario where A and B are communicating with each other



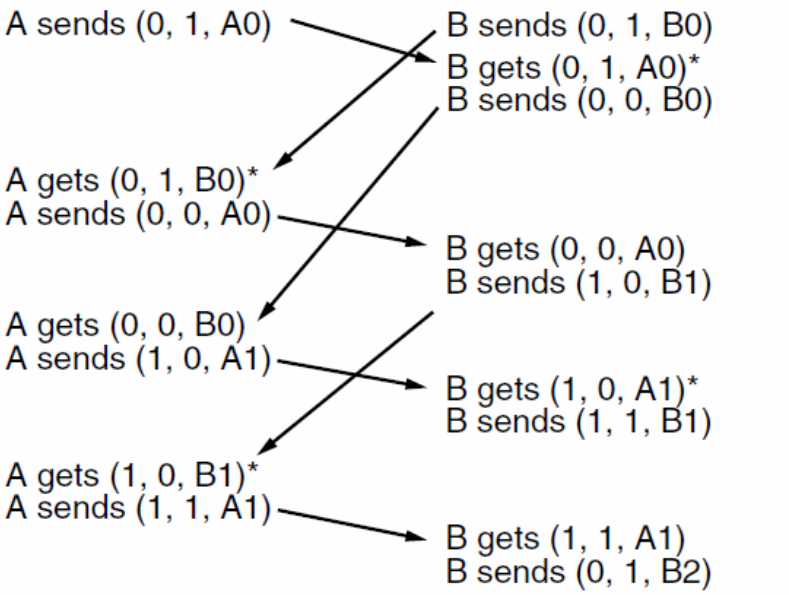
1. A sends its first frame :
   A sends (0, 1, A0)
2. B receives A's :
   B gets (0,1,A0)*
   B sends its own data and ACK : B sends (0, 0, B0)
   Seq = 0 → B's own data is B0
   ACK = 0 → B acknowledges A's frame 0
   B0 → B's data
   So one frame performs **two jobs**:Send B's data + acknowledge A's data. This is called **piggybacking**.
3. A receives B's frame :
   A gets (0,0,B0)* - B has correctly received my A0.
   A sends (1,0,A1) - A can now send its next data frame The sequence number changes from **0 → 1**.

4. If **B has no data to send**, B still needs to **acknowledge A's frame**. In that case, B sends a **control/ACK frame without data**. Example:
   A send data to B : A → B : (0, 1, A0)
   B receives A0 correctly, but B has no data of its own. So B sends: B → A : (—, 0, ACK)
   Where 0 in the ACK field means **B has received A's frame 0**. There is **no B data** in this frame.

Thus Each frame contains: **"My sequence number + Your last successfully received frame + My data."** Therefore, **both A and B can communicate simultaneously while acknowledging each other's frames**, which makes the one-bit sliding window protocol efficient for bidirectional communication

Example 2: case where A and B start communication simultaneously

In this case **there is no transmission error**, but duplicate frames are generated because **A and B started at exactly the same time**. When A and B start simultaneously, their initial frames cross. This can cause both sides to retransmit their initial frames, producing duplicate frames even though the communication channel has no errors. The one-bit sequence number allows the receiver to recognize these duplicates and avoid delivering the same data again.



synchronization difficulty of the one-bit sliding-window protocol.

Maximum window size w in Sliding Window protocol

In a sliding-window Link utilization is given by the formula

$$\text{link utilization} \leq \frac{w}{1+2BD}$$

Where:

1. Bandwidth-Delay (DB) is used to determine the required window size.

The bandwidth-delay product (BDP) tells us how much data can be "in transit" on the link at one time

$$BDP = \text{Bandwidth} \times \text{One-way propagation delay}$$

To express this in **frames**:

$$BD = \frac{\text{Bandwidth} \times \text{One-way delay}}{\text{Frame size}}$$

For the example suppose we have bandwidth of 50 kbps and One-way delay = 250 ms and Frame size of 1000 bits

So:

$$BD = \frac{50,000 \times 0.25}{1000}$$

$$BD = 12.5 \text{ frames}$$

2. Maximum window size (W):

The sender has to keep sending frames while waiting for acknowledgements. Because an acknowledgement requires a **round trip**, the sender must be able to have approximately 2BD. frames outstanding. The extra +1 accounts for the fact that an acknowledgement is generated only **after the complete frame has arrived**. Therefore**: w=2BD+1**

For the above example: w=2(12.5)+1

w=25+1=26

For 100% Utilization of the channel sender should send frame with the gap of given by framesize / bandwidth

Example: Each frame contains 1000 bits, and the bandwidth is 50 kbps frame delay is given by:

$$\frac{1000}{50,000} = 0.02 \text{ sec} = 20 \text{ ms}$$

Therefore, the sender can transmit **one frame every 20 ms**. It sends 26 frames in: 26×20=520 ms. The acknowledgement for frame 0 arrives at about **520 ms**, exactly when the sender needs permission to send the next frame. After that, acknowledgements arrive every **20 ms**, matching the sender's transmission rate. Hence, the sender never has to wait, giving Utilization=100%

utilization of the link can be expressed as: If w is large enough to keep the channel continuously busy:

$$U = \frac{w}{2BD+1}$$

Example: Suppose we have

1. Bandwidth = 50Mbps (50,000 bits/sec)
2. One-way delay = 250 ms (0.25 sec)

3. Each frame = 1,000 bits

1. First calculate how many bits can be in the channel:
   Bandwidth-Delay Product = Bandwith * one way delay
   = 50,000×0.25=12,500 bit
2. Convert bits into **frames**:

$$BD = \frac{\text{Bandwidth} \times \text{Delay}}{\text{Frame size}} \qquad BD = \frac{12,500 \text{ bits}}{1,000 \text{ bits/frame}} = 12.5 \text{ frames}$$

This technique allows the sender to transmit multiple packets before requiring an acknowledgment. However, if an acknowledgment is not received within a specified time, the sender retransmits all unacknowledged packets from the last correctly received acknowledgment. This technique is suitable for scenarios with low error rates

Characteristics of Go-Back-N (GBN)

Sender Window Size (Ws) : In Go-Back-N, the sender window size (Ws) is equal to N. For example, in GBN-10, the sender can transmit 10 frames without waiting for acknowledgements.The value of N must be greater than 1 to enable pipelining. When N = 1, Go-Back-N behaves like the Stop-and-Wait ARQ protocol.

Receiver Window Size (WR): Receiver window size (WR) is always 1.Receiver accepts only the next expected frame.Frames arriving out of order are discarded.Receiver does not buffer out-of-order frames.

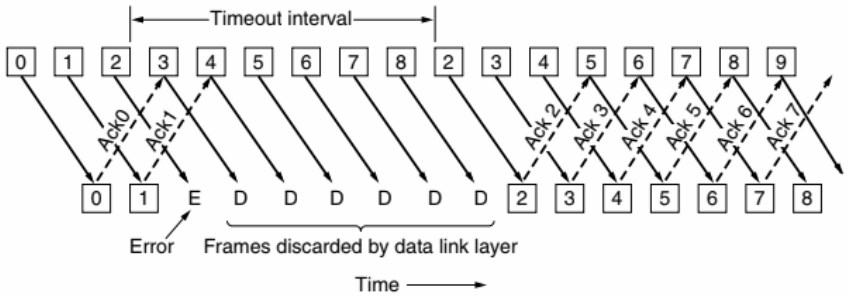
Acknowledgements : Acknowledgments are control messages sent by the receiver to confirm the successful reception of data frames. They play a crucial role in ensuring reliable communication. If the sender does not receive an acknowledgment within a specified timeout interval, it assumes the frame was lost or corrupted and retransmits it.

A **cumulative ACK** confirms that all frames up to a specific sequence number have been received successfully. It is used in **Go-Back-N (GBN)** to reduce ACK traffic, though a lost ACK may cause unnecessary retransmissions.

Working of GB-N Protocol

Sender Side

- The sender maintains a sliding window of size N (e.g., in GBN-4, the window size is 4).
- It can transmit up to N frames without waiting for individual ACKs.
- Each transmitted frame is associated with a timer.
- When an ACK is received, the sender slides the window forward, allowing transmission of new frames.
- If a timeout occurs for a frame, the sender retransmits that frame and all subsequent frames in the current window.



Receiver Side

- The receiver maintains a window size of 1 (WR = 1).
- It accepts only the next expected frame in sequence.
- If the expected frame is received correctly, then the receiver sends an ACK, and updates the expected sequence number.
- If an out-of-order frame is received, then the receiver discards the frame, and resends the ACK for the last correctly received frame.

Selective Repeat:

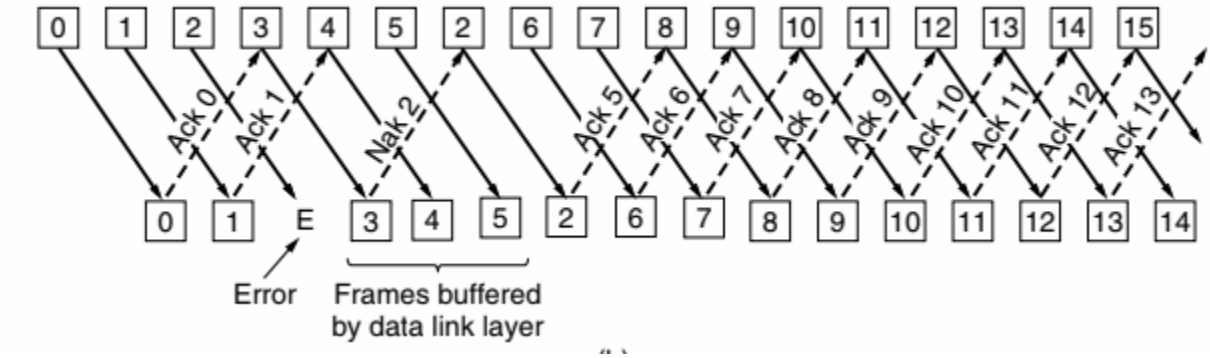
Selective repeat protocol, also known as **Selective Repeat Automatic Repeat Request** (ARQ), is a data link layer protocol that uses the sliding window technique for reliable data frame delivery. Only erroneous or lost frames are retransmitted in this case, while good frames are received and buffered.

In this technique, the receiver individually acknowledges each correctly received packet, while requesting retransmission of only the lost or corrupted ones. The sender stores and retransmits only the specific packets requested by the receiver, leading to efficient bandwidth utilization

The other general strategy for handling errors when frames are pipelined is called selective repeat. When it is used, a bad frame that is received is discarded,but any good frames received after it are accepted and buffered. When the sender times out, only the oldest unacknowledged frame is retransmitted.

- In Selective Repeat ARQ, only the erroneous or lost frames are retransmitted, while correct frames are received and buffered.
- While keeping track of sequence numbers, the receiver buffers the frames in memory and sends **NACK** (negative acknowledgement) for only the missing or damaged frame.
- The sender will send/retransmit the packet for which NACK (negative acknowledgement) is received.

Example:
Suppose there is a sender and a receiver. There are 11 frames to be sent, and the frames are numbered as 0,1,2,3,4,5,6,7,8,9,10. The size of the window N decides the sequence number of the frames. For better understanding, we have taken the running numbers. Let's take the sender's window size to be 4, which means the sender can send four frames before expecting any acknowledgement from the first frame, which is 0.



Difference between Go-Back-N and Selective Repeat

Go-Back-N (GBN)	Selective Repeat (SR)

Retransmits the lost/damaged frame **and all subsequent frames**.	Retransmits **only the lost/damaged frame**.
Uses **cumulative ACKs** and generally does not require NAK.	Uses **individual ACKs and NAKs** to identify missing frames.
Receiver discards out-of-order frames.	Receiver **stores out-of-order frames**.
Requires **less receiver memory**.	Requires **more receiver memory**.
Simpler but causes **more retransmissions**.	More complex but **more efficient** when errors occur.